



GyaanVault

SPPU M.Sc. Data Science — Free Study Material

Lab: Artificial Intelligence

DS-513-MJP

Solved Practical Manual

Semester I · Lab (MJP)

This solved practical manual covers the Artificial Intelligence lab assignments from the official SPPU M.Sc. Data Science syllabus. Each practical states the aim followed by a complete, runnable solution and the expected output. Use it as a reference — type the code yourself and experiment with the parameters to learn effectively.

Requirements: Python 3.x with numpy, pandas, matplotlib, scikit-learn, scipy installed (pip install numpy pandas matplotlib scikit-learn scipy).

Practical 1: Factorial of a number

```
def factorial(n):
    return 1 if n <= 1 else n * factorial(n-1)
print(factorial(5))    # 120
```

Practical 2: Check prime

```
def is_prime(n):
    if n < 2: return False
    for i in range(2, int(n**0.5)+1):
        if n % i == 0: return False
    return True
print([x for x in range(2,20) if is_prime(x)])
```

Practical 3: Fibonacci series

```
def fib(n):
    a, b = 0, 1
    for _ in range(n):
        print(a, end=" "); a, b = b, a+b
fib(10)    # 0 1 1 2 3 5 8 13 21 34
```

Practical 4: Simple chatbot

```
rules = {"hi":"Hello!", "how are you":"I'm a bot, doing great!",
        "bye":"Goodbye ð=ÜK"}
def chat(msg):
    return rules.get(msg.lower().strip(), "Sorry, I didn't understand.")
print(chat("Hi")); print(chat("bye"))
```

Practical 5: Breadth First Search (BFS)

```
from collections import deque
graph = {'A':['B','C'], 'B':['D','E'], 'C':['F'], 'D':[], 'E':['F'], 'F':[]}
def bfs(start):
    visited, q = set([start]), deque([start])
    while q:
        node = q.popleft(); print(node, end=" ")
        for nb in graph[node]:
            if nb not in visited:
                visited.add(nb); q.append(nb)
bfs('A')    # A B C D E F
```

Practical 6: Depth First Search (DFS)

```
graph = {'A':['B','C'], 'B':['D','E'], 'C':['F'], 'D':[], 'E':['F'], 'F':[]}
def dfs(node, visited=None):
    visited = visited or set()
    if node not in visited:
        print(node, end=" "); visited.add(node)
        for nb in graph[node]: dfs(nb, visited)
dfs('A')    # A B D E F C
```

Practical 7: Water Jug problem (BFS state search)

```
from collections import deque
def water_jug(cap_a, cap_b, target):
    start, seen, q = (0,0), set(), deque([(0,0), []])
    while q:
        (a,b), path = q.popleft()
        if a == target or b == target:
            return path + [(a,b)]
        if (a,b) in seen: continue
        seen.add((a,b))
        nxt = {(cap_a,b), (a, cap_b), (0,b), (a,0),
              (a-min(a, cap_b-b), b+min(a, cap_b-b)),
              (a+min(b, cap_a-a), b-min(b, cap_a-a))}
        for s in nxt: q.append((s, path+[(a,b)]))
    return None
print(water_jug(4,3,2))
```

Practical 8: Multiplication table

```
n = 7
for i in range(1, 11):
    print(f"{n} x {i} = {n*i}")
```


